Tuning Hashes

Selected Projects in Performance Engineering of
Hash-Based Data Structures and Hash Functions

Calvin James Higgins
Department of Computer Science and Statistics
University of Rhode Island
April 12, 2026

A portfolio presented for the degree of
Master of Science in Computer Science

1 Introduction

The projects in this portfolio grew out of self-directed study. Unlike my published research, which developed within an established advising relationship and with relevant coursework as a foundation, this work required me to acquire the necessary background, conduct literature reviews, and define and pursue research questions independently. Each project addresses a distinct problem in performance engineering of hash-based data structures and hash functions. For each, I generalized a restrictive parameterization, then automatically optimized within the broader space to improve performance.

The first generalizes binary fuse filters, a recent approximate set membership structure, to support non-integer arity[1] natively. Binary fuse filters offer only a discrete tradeoff: increasing arity reduces memory overhead but increases query time. The accepted workaround partitions data between disjoint filters of different arity to achieve a continuous tradeoff. Native non-integer arity makes this principled, and implementations found by large-scale search require less memory than the partitioning approach.

The second introduces the first optimal parameter tuning algorithm for secure learned Bloom filters, a recent approximate set membership structure that combines a learned model with two secure filters. Since no tuning algorithms were known, the recommended approach was simply to allocate memory evenly between the two filters. This default cannot target an expected false positive rate, nor bound the worst-case rate, and wastes memory when either constraint is defined. I give the first exact tuning algorithm to minimize memory footprint subject to expected and maximum false positive rate constraints.

The third extends entropy-aware hash function specialization, a technique to select just enough bytes to form a good enough hash. Previous cost models ignored key frequency, target data structure and target architecture, leading to incomplete specialization and suboptimal performance. I generalize cost models to incorporate these factors, then demonstrate the approach by specializing for `std::unordered_set` with URL keys on the `x64` platform. The resulting hash function reduces both hash and hash table query latency.

Together these projects reflect a single approach to performance engineering: define expressive parameter spaces, and automatically optimize within them. Each project required independently building fluency in an unfamiliar area, and each left open problems I continue to pursue. They are the clearest evidence that I have become a computer scientist.

[1] The number of hash functions per query.

2 Non-Uniform Binary Fuse Filters

This project was completed for CSC 592: Algorithms for Big Data.

2.1 Motivation

Distributed systems often require fast and small set data structures. For example, content delivery networks avoid caching one-hit wonders by querying a visited set [12]. Supporting such exact membership queries on a large universe[2] U requires $\Omega(\lg |U|)$ bits per key. For large enough sets, this is impractical. By permitting false positive results with some probability ϵ, the lower bound is reduced to $\lg \frac{1}{\epsilon}$ bits per key [3]. In other words, *approximate* membership queries can be much more memory efficient than exact membership queries [2, 7, 6].

A binary fuse filter [6] is a recent static probabilistic data structure for approximate membership queries. Membership information is stored in an array of m-bit values partitioned into disjoint segments. To query the filter, a key is hashed to a fingerprint value and to three array entries located in consecutive segments (see Figure 1). If the bitwise-XOR of the entries is equal to the fingerprint, then the query *might* be in the set. Otherwise, the query *definitely* is not in the set.

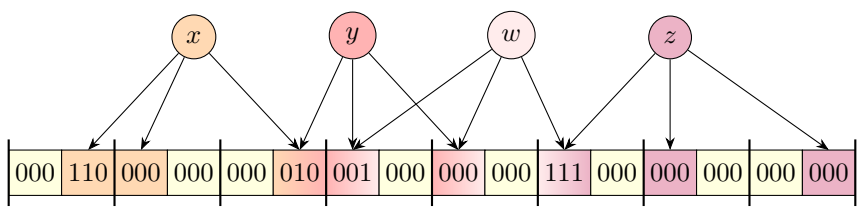

Figure 1: *Binary fuse filter with 16-entry array, 2-entry segments, and 3-bit fingerprints. Vertical bars mark segment boundaries. The filter contains keys* x, y, w *and* z, *with fingerprints* 100, 011, 110 *and* 111, *respectively. Each key hashes to three consecutive segments. Corresponding entries are marked by arrows and matching colors. For example, since the filter contains* x, *the bitwise-XOR* $110 \oplus 000 \oplus 010$ *is the fingerprint* 100.

To construct a binary fuse filter, choose each array entry such that, for each key, the bitwise-XOR of the three corresponding array entries is equal to the fingerprint. This requires solving a system of linear equations with variables in $\mathbb{F}_2^m$ (i.e. m-bit vectors) where addition is performed using bitwise-XOR. Let h_1, h_2 and h_3 be the hash functions, f be the fingerprint function, and A be the array. Each key k yields a linear equation

$$A[h_1(k)] \oplus A[h_2(k)] \oplus A[h_3(k)] = f(k)$$

[2] Set of all keys.

that must be satisfied (see Figure 2). If some equation is not satisfied, the filter will incorrectly report the corresponding key is *definitely* not in the set. Since approximate membership queries allow only false positives and not false negatives, the choice of array entries must satisfy every equation.

$$A[1] \oplus A[2] \oplus A[5] = 100 \qquad A[5] \oplus A[6] \oplus A[8] = 011$$

$$A[6] \oplus A[8] \oplus A[10] = 110 \qquad A[10] \oplus A[12] \oplus A[15] = 111$$

Figure 2: *System of linear equations with variables in $\mathbb{F}_2^3$ induced by Figure 1. Each $A[i]$ is the i-th entry in the backing array A (zero-indexed). The keys are x, y, w and z with fingerprints 100, 011, 110 and 111, respectively. Array entries and fingerprints are marked with corresponding key color(s). There is one equation for each key.*

To solve the system, model it as a hypergraph. A hypergraph is a graph where edges may contain any subset of vertices. For each entry, create a vertex. For each equation, create an edge containing entries (vertices) in the equation (see Figure 3). Then, 2-peel the hypergraph, that is:

1. Remove vertices of degree zero, then set their entries to 0^m.
2. Remove vertices of degree one and their incident edges. Then, set their entries to the bitwise-XOR of the fingerprint and the two other entries in the edge. If the other entries are not yet set, defer until they are set.
3. Repeat until there are no vertices of degree less than two, or until the empty hypergraph is reached (i.e. all entries set).

If the empty hypergraph is reached, the hypergraph is called 2-peelable. The system is solved if and only if its hypergraph is 2-peelable.

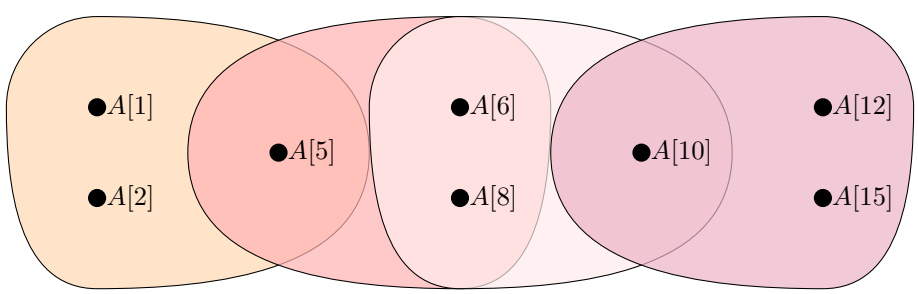

Figure 3: *Hypergraph model of Figure 2. Each $A[i]$ is the i-th entry in the backing array A (zero-indexed). Hyperedges correspond to keys x, y, w and z, colored accordingly. Vertices of degree zero not shown.*

In the filter setting, edges represent keys and vertices represent array entries. Lower edge size means cheaper queries, since fewer array entries (vertices) are accessed per key (edge). Higher edge density means lower space usage, since there are more keys (edges) per array entry (vertex).

Many hypergraph families exhibit sharp phase transitions in peelability [4, 14]. When the edge density is below a critical value, called the 2-peelability threshold, sampled hypergraphs are almost surely 2-peelable. When the density exceeds this threshold, they are almost surely not. The threshold is therefore a direct measure of space efficiency. A higher threshold means more keys can be stored per array entry.

Binary fuse filters induce the fuse graph family, which achieves the supremum 2-peelability threshold among 3-uniform hypergraph[3] families [14]. This threshold occurs at an edge density of 0.918, or roughly $1.089 \lg \frac{1}{\epsilon}$ bits per key [14]. No family of 3-uniform hypergraphs has higher 2-peelability threshold. More generally, a binary fuse filter with $m \geq 3$ hash functions[4] saturates the 2-peelability threshold for m-uniform hypergraphs [14].

However, this optimality result is confined to uniform hypergraphs. There may exist families of non-uniform hypergraphs with higher 2-peelability thresholds. If such families also have a low average edge size (i.e. few hashes and accesses per query), filters based on them could potentially reduce memory overhead without degrading query latency and throughput. Therefore, we ask:

1. How can we systematically construct non-uniform hypergraph families with peelability thresholds comparable to or surpassing those of fuse graph families?

2. How can we design efficient approximate membership query data structures by exploiting peelable non-uniform hypergraph families?

2.2 Summary

2.2.1 Non-Uniform Hypergraphs

We computed 2-peelability thresholds for 10K non-uniform fuse graph families with mean edge size between 3 and 4 (see Figure 4). As peelability is monotonic in edge density, we used binary search. At each candidate threshold, we sampled three hypergraphs with 1M edges. If the majority were peelable, we considered the family peelable at that threshold. We implemented this algorithm in C++ and found thresholds in parallel on a compute cluster.

[3] In general, a hypergraph is m-uniform if every edge contains exactly m vertices.

[4] Only binary fuse filters with three or four hash functions are practical. With four hash functions, the 2-peelability threshold is already 0.977, yielding about $1.023 \lg \frac{1}{\epsilon}$ bits per key or within 2.3% of optimal [14].

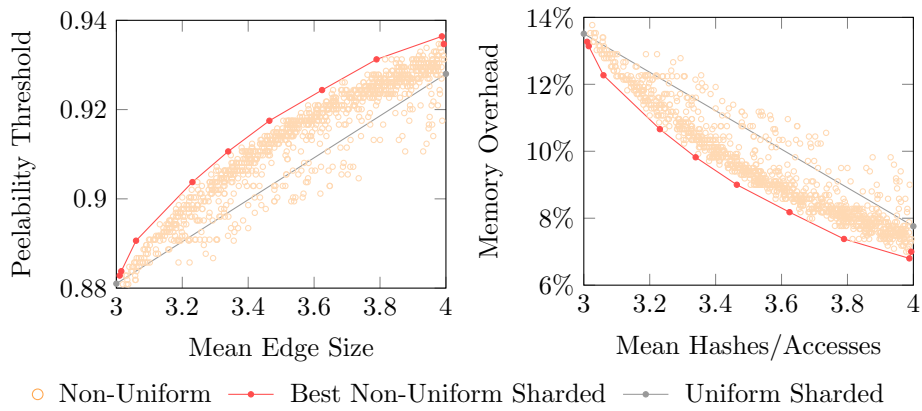

Figure 4: *Empirical peelability thresholds and memory overheads for non-uniform fuse graphs with 1M edges and mean edge size between 3 and 4. Orange dots show 1K subsampled families. Red shows the best non-uniform sharded families. Gray shows the sharded uniform baseline. Many non-uniform families outperform the uniform baseline. Left: mean edge size vs. peelability threshold (higher is better). Right: theoretical query cost vs. memory overhead (lower is better).*

While m-uniform graphs have integer mean edge sizes, they can be extended to any mean edge size via sharding[5] [6]. For example, splitting edges evenly between edge-disjoint 3-uniform and 4-uniform subgraphs yields a mean edge size of 3.5. Many non-uniform families outperform this baseline, and therefore afford smaller filters. Interestingly, some non-uniform families also outperform uniform families at integer mean edge sizes. Since peelability thresholds are defined in the limit of large graphs, a non-uniform family can be more peelable in practice than a uniform family with nearly the same mean edge size. It's also possible three samples per candidate is not enough to accurately determine the threshold. For example, one family achieves $\approx 1\%$ memory savings over the uniform baseline at mean edge size 4.

These data suggest two conjectures. Let $\mathcal{H}_d$ denote the set of fuse graph families with mean edge size d and let $\tau(\mathcal{F})$ denote the 2-peelability threshold of a family $\mathcal{F}$. Define $\Gamma(d) = \sup\{\tau(\mathcal{F}) : \mathcal{F} \in \mathcal{H}_d\}$. We conjecture that $\Gamma(d)$ is continuous, monotonically increasing, and concave. Since a concave Γ would lie above any linear interpolation, this implies that sharding is suboptimal. We further conjecture that for $d \notin \mathbb{Z}$, the supremum is attained by the family with a fraction $(1-p)$ of $\lfloor d \rfloor$-edges and a fraction p of $\lceil d \rceil$-edges, where $p = d - \lfloor d \rfloor$. For example, for $d = 3.5$, this family contains 50% 3-edges and 50% 4-edges. Both conjectures remain open.

[5] Sharding partitions a dataset into disjoint subsets. For hypergraphs, each subset becomes an edge-disjoint subgraph. For filters, each subset is stored in a separate filter.

2.2.2 Non-Uniform Binary Fuse Filters

We implemented and optimized a non-uniform binary fuse filter with 50% 3-edges and 50% 4-edges in C++ (i.e. each key hashes to either 3 or 4 array entries). This edge distribution offered the greatest advantage over sharding uniform binary fuse filters. We considered three query algorithms: conditional branch, branchless and phantom entry described below (see Table 1 for metrics and Section A.2 for pseudocode).

Algorithm	Hashes	Branch Mispredictions	Cache Misses
Conditional Branch	3.5	0.5	3.5
Branchless	4	0	4
Phantom Entry	4	0	3.5

Table 1: *Metrics for non-uniform binary fuse filter query algorithms. Hashes, branch mispredictions, and cache misses are mean per query.*

In all three algorithms, we report that the key *might* be in the set if the XOR of the accessed array entries equals the fingerprint. The algorithms differ only in how they handle variable edge size when computing the XOR.

In the conditional branch algorithm (see Figure 2), we XOR the first three array entries. If the key is a 4-edge, we XOR in the fourth entry. This branch mispredicts on at least half of all queries.

In the branchless algorithm (see Figure 3), we load all four array entries unconditionally. If the key is a 3-edge, we zero the fourth value with a mask before XORing all four values. For large filters, all four loads will miss the cache.

In the phantom entry algorithm (see Figure 4), we load all four array entries unconditionally. However, if the key is a 3-edge, we zero the fourth hash address before the load, redirecting it to $A[0]$. To support this, we shift all valid entries left by one index and set $A[0] = 0$ at construction time (see Figure 5). Unlike the branchless algorithm, the fourth load for 3-edges hits the cache, since $A[0]$ is accessed in 50% of queries.

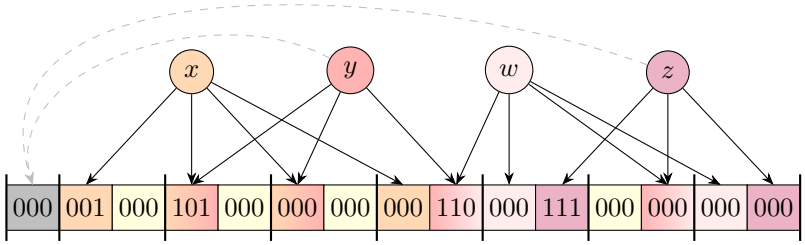

Figure 5: *Non-uniform binary fuse filter with phantom entry, 14-entry array, 2-entry segments, and 3-bit fingerprints. Vertical bars mark segment boundaries. The phantom entry is colored gray. The filter contains keys* x, y, w *and* z, *with fingerprints* 100, 011, 110 *and* 111, *respectively. Each key hashes to three or four consecutive segments. Corresponding entries are marked by arrows and matching colors. Dashed gray arrows indicate hashes to the phantom entry. For example, since the filter contains* y, *the bitwise-XOR* $101 \oplus 000 \oplus 110 \oplus 000$ *is the fingerprint* 011. *See Figure 9 and Figure 10 for the induced system of equations and hypergraph.*

Surprisingly, only the conditional branch algorithm achieved higher query throughput than the 4-uniform binary fuse filter, and even this fell short of an acceptable time-space tradeoff. Since we expected nearly all cycles to be spent stalled on memory loads, reducing cache misses per query should have improved throughput proportionally. In practice, however, only 80% of cycles were stalled on loads, and there were no other stalls.

We hypothesize that out-of-order (OoO) execution stalls before it can compute load addresses for the next key. As a result, when outstanding loads complete, hashes for the next query have not yet been computed, and the subsequent loads are delayed by the full hash latency. This dependency also explains the conditional branch algorithm's advantage: its lower hash latency reduces this delay.

The root cause is likely one of two structural hazards: either a shortage of miss status holding registers (MSHRs) prevents prefetching the next key (and therefore precomputing its hashes), or the reorder buffer (ROB) is too small to precompute enough hashes for future keys. Based on microarchitectural parameters and the benchmark assembly, the RoB should be sufficiently large so the first explanation is more likely. This indicates that the benchmarking harness, not non-uniformity, is the primary bottleneck.

2.2.3 Future Work

We see four promising directions for future work. First, batching may yield a non-uniform binary fuse filter with a competitive time-space tradeoff. Second, interleaving the segments of a binary fuse filter could improve cache locality during queries. Third, introducing additional weak points into the fuse graph would

enable greater parallelism during filter construction. Currently, the fuse graph has only two weak points, one at each end, limiting peeling to two threads. The recommended workaround is sharding, but this may sacrifice memory efficiency. Fourth, we aim to prove the conjectures stated in Section 2.2.1, for which the methods of [4] and [14] may be useful.

2.3 Reflection

This project strengthened my ability to enter a new subfield independently through a literature review. Reading many approximate membership query papers, from theoretical analysis to engineered implementations, gave me enough grounding to pose original research questions. One area I want to develop further is posing robust questions, where even a negative result is informative. If non-uniform filter's microarchitectural bottleneck cannot be overcome, that facet is a total loss.

The most demanding aspect of this project was maintaining multiple representations of the same structure simultaneously: a filter is a data structure, a system of linear equations, and a hypergraph. Initially, I found it challenging to move fluidly between these views. Over time this became natural, and I applied it most directly when using cluster resources to empirically explore non-uniform hypergraphs, and translate the result into both formal conjectures and a concrete implementation.

The performance engineering work expanded my understanding of modern processor architecture. Attempting to make the non-uniform filter practical required reasoning carefully about superscalar out-of-order execution and non-blocking caches, interpret hardware performance counters, and diagnose microarchitectural bottlenecks.

Finally, I want to thank Morgan, Tom, Vincent and Dan. Collaborating with them is definitely a highlight of my academic experience.

3 Tuning Secure Learned Bloom Filters

This project was completed for CSC 592: Machine Learning Security.

3.1 Background

Database systems often require fast and small set data structures. For example, RocksDB [5] reduces IO operations by filtering queries before reading from disk. As false positives do not impact correctness, they are permitted with bounded probability. Such approximate membership queries can be much faster and more memory efficient than exact membership queries.

A Bloom filter [2] is a probabilistic data structure for approximate membership queries. Membership information is stored in an array of bits. To query the filter, a key is hashed to several entries in the array (see Figure 6). If all entries are set to 1, then the key *might* be in the set, and false positives occur with probability ϵ. Otherwise, the query *definitely* is not in the set.

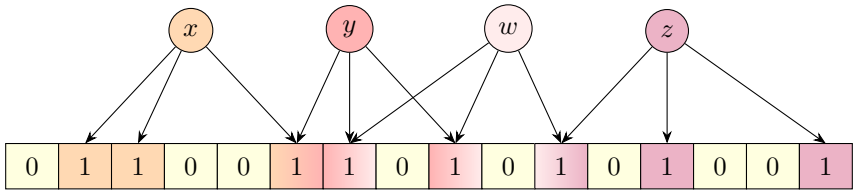

Figure 6: *Bloom filter with 16-entry array and 3 hash functions. The filter contains keys* x, y, w *and* z. *Corresponding entries are marked by arrows and matching colors. For example, since the filter contains* x, *the bitwise-AND* $1 \,\&\, 1 \,\&\, 1$ *is* 1.

Learned Bloom filters [10] extend Bloom filters with a machine learning model that predicts set membership (see Figure 8). For many practical datasets, the model compresses membership information more efficiently than a Bloom filter, reducing memory consumption. Queries are first checked with the model. On a positive prediction, the key may or may not be in the set, so false positives occur at the model's false positive rate. On a negative prediction, the key may still be in the set due to model false negatives, so a backup Bloom filter storing all false negatives is queried. False positives then occur at the product of the model's true negative rate and the backup filter's false positive rate. The overall false positive rate is the sum of these two terms.

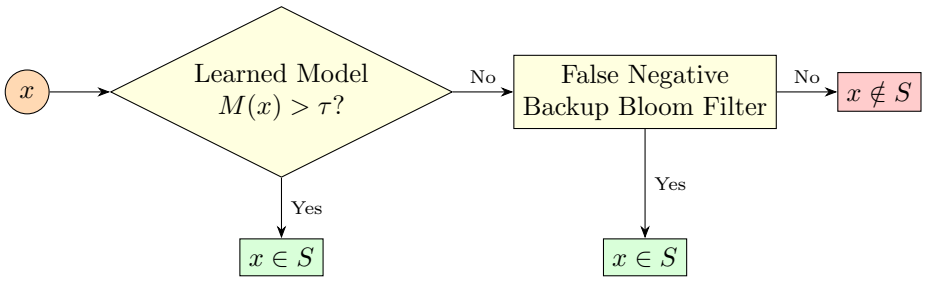

Figure 7: *Learned Bloom filter. Learned model M is queried with key x. If M predicts membership with score greater than τ, output that x is in the set. Otherwise, output the result of querying a Bloom filter containing the false negatives of M. False positives are possible but not false negatives.*

Since standard machine learning models are vulnerable to adversarial attack, an attacker can generate misclassified non-members and arbitrarily degrade the

false positive rate. Secure learned Bloom filters [1] defend against this by adding a second backup Bloom filter to confirm positive predictions against all true set members, rejecting model false positives. However, the backup filters are vulnerable to attack. Secure Bloom filters[6] [1] address this by applying a pseudorandom permutation to keys before querying, ensuring no efficient adversary can non-negligibly degrade the false positive rate (assuming $\mathbf{P} \neq \mathbf{NP}$). Secure learned Bloom filters therefore replace both backup filters with secure Bloom filters. The false positive rate is then bounded by the maximum of the two backup filters' false positive rates.

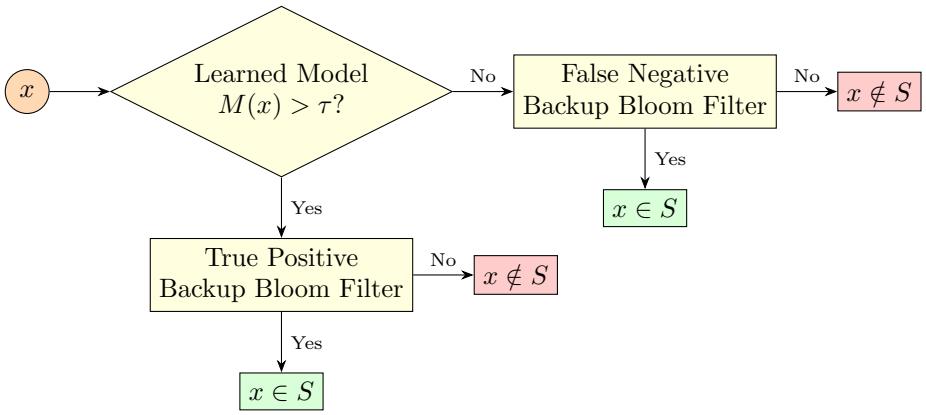

Figure 8: *Learned Bloom filter. Learned model M is queried with key x. If M predicts membership with score greater than τ, output the result of querying a Bloom filter containing true positives of M. Otherwise, output the result of querying a Bloom filter containing the false negatives of M. False positives are possible but not false negatives.*

Like learned Bloom filters, several secure learned Bloom filter parameters must be tuned to attain desired expected and worst-case false positive rates, including the decision threshold and backup filter false positive rates. These parameters also control memory consumption, so they must be set carefully to preserve memory efficiency. Tuning procedures for learned Bloom filters do not directly translate to secure learned Bloom filters, because they only optimize for expected false positive rate, ignoring the worst-case guarantee. Therefore, we ask:

1. How can the decision threshold and backup filter false positive rates be chosen to minimize memory footprint given desired expected and maximum false positive rates?

2. How well do secure learned Bloom filters perform in practice with and without parameter tuning?

[6]Secure Bloom filters are often called Naor-Eylon filters.

3.2 Summary

3.2.1 Tuning Algorithm

Given a desired expected false positive rate ϵ and maximum false positive rate $\epsilon_{\max}$, we wish to set the decision threshold τ and backup filter false positive rates such that the filter size is minimized. We proceed in two steps. First, fix the decision threshold, and find backup filter false positive rates that yield the smallest filter obeying the expected false positive rate. Second, choose the decision threshold that yields the smallest filter obeying the maximum false positive rate.

The total size of a secure learned Bloom filter in bits is

$$-\frac{n}{\ln(2)^2}\left(\mathrm{M}_{\mathrm{TPR}}\ln(\mathrm{TP}_{\mathrm{FPR}})+\mathrm{M}_{\mathrm{FNR}}\ln(\mathrm{FN}_{\mathrm{FPR}})\right)+\mathrm{M}_{\mathrm{bits}}$$

where n is the set's size, M is the model, TP is the true positive filter and FN is the false negative filter. $\mathrm{M}_{\mathrm{bits}}$ is the memory consumption of the learned model in bits. Subscripts on M denote model rates, and subscripts on TP and FN denote backup filter false positive rates.

Given a fixed decision threshold, we prove the total size is minimized subject to expected false positive rate ϵ by taking

$$\mathrm{TP}_{\mathrm{FPR}}=\epsilon\frac{\mathrm{M}_{\mathrm{TPR}}}{\mathrm{M}_{\mathrm{FPR}}} \qquad \mathrm{FN}_{\mathrm{FPR}}=\epsilon\frac{\mathrm{M}_{\mathrm{FNR}}}{\mathrm{M}_{\mathrm{TNR}}}$$

where M is the model, TP is the true positive filter, and FN is the false negative filter. Subscripts denote false positive, false negative, true positive, and true negative rates.

Then, we show that the optimal choice of decision threshold minimizes

$$C(\tau)=-\mathrm{M}_{\mathrm{TPR}}\ln\left(\frac{\mathrm{M}_{\mathrm{TPR}}}{\mathrm{M}_{\mathrm{FPR}}}\right)-\mathrm{M}_{\mathrm{FNR}}\ln\left(\frac{\mathrm{M}_{\mathrm{FNR}}}{\mathrm{M}_{\mathrm{TNR}}}\right)$$

To maintain worst-case security guarantees (i.e. bounded maximum false positive rate), we restrict the choice of τ such that

$$\max\left\{\epsilon\frac{\mathrm{M}_{\mathrm{TPR}}}{\mathrm{M}_{\mathrm{FPR}}},\epsilon\frac{\mathrm{M}_{\mathrm{FNR}}}{\mathrm{M}_{\mathrm{TNR}}}\right\}\leq\epsilon_{\max}$$

so the false positive rate remains bounded even if the adversary forces all-positive or all-negative predictions. The optimal τ can be found in $\Theta(n\lg n)$ time (see Algorithm 1).

Algorithm 1 Optimal threshold selection

function FindThreshold($\hat{Y}$, Y, ϵ, $\epsilon_{\max}$)
  Sort confidence scores $\hat{Y}$ in ascending order and reindex labels Y
  // n is set size, P is positive count, N is negative count
  $n \leftarrow |Y|, \quad P \leftarrow \sum_i Y_i, \quad N \leftarrow n - P$
  // τ^* and C^* are optimal threshold and cost.
  $\tau^* \leftarrow \varnothing, \quad C^* \leftarrow \infty$
  // TP_i and FP_i are true and false positive counts
  $\mathrm{TP}_i \leftarrow P, \quad \mathrm{FP}_i \leftarrow N$
  // On iteration i, $\hat{Y}_j$ for $1 \leq j \leq i$ are negative and rest are positive
  for $i = 0$ **to** n **do**
    // Find threshold to set $\hat{Y}_j$ for $1 \leq j \leq i$ negative and rest positive

$$\tau_i \leftarrow \begin{cases} 0 & i = 0 \\ 1 & i = n \\ \left(\hat{Y}_i + \hat{Y}_{i+1}\right)/2 & \text{otherwise} \end{cases}$$

    // FN_i and TN_i are false and true negative counts
    $\mathrm{FN}_i \leftarrow P - \mathrm{TP}_i, \quad \mathrm{TN}_i \leftarrow N - \mathrm{FP}_i$
    // Find maximum false positive rate ϵ_i for optimal with threshold τ_i
    $\mathrm{M}_{\mathrm{TPR}} \leftarrow \mathrm{TP}_i/P, \quad \mathrm{M}_{\mathrm{FPR}} \leftarrow \mathrm{FP}_i/N$
    $\mathrm{M}_{\mathrm{FNR}} \leftarrow \mathrm{FN}_i/P, \quad \mathrm{M}_{\mathrm{TNR}} \leftarrow \mathrm{TN}_i/N$

$$\epsilon_i \leftarrow \begin{cases} \infty & \mathrm{M}_{\mathrm{FPR}} = 0 \text{ or } \mathrm{M}_{\mathrm{TNR}} = 0 \\ \max\left\{\epsilon \dfrac{\mathrm{M}_{\mathrm{TPR}}}{\mathrm{M}_{\mathrm{FPR}}}, \epsilon \dfrac{\mathrm{M}_{\mathrm{FNR}}}{\mathrm{M}_{\mathrm{TNR}}}\right\} & \text{otherwise} \end{cases}$$

    // Maintain optimal threshold and cost
    if $\epsilon_i \leq \epsilon_{\max}$ **and** $C(\tau_i) < C^*$ **then**
      $\tau^* \leftarrow \tau_i, \quad C^* \leftarrow C(\tau_i)$
    end if
    // Maintain true and false positive counts
    $\mathrm{TP}_i \leftarrow \mathrm{TP}_i - Y_i, \quad \mathrm{FP}_i \leftarrow \mathrm{FP}_i - (1 - Y_i)$
  end for
  return τ^*
end function

3.2.2 Tuned Performance

We construct a secure Bloom filter and a secure learned Bloom filter from a dataset of malicious and benign URLs[7]. For the same memory budget and maximum false positive rate, the tuned secure learned Bloom filter reduces the expected false positive rate by half relative to the secure Bloom filter (9.7% vs. 4.6%). We verify our implementation using several adaptive adversarial attacks, and find none increase the false positive rate above $\epsilon_{\max}$.

[7] `https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset`

3.2.3 Future Work

We see three promising directions for future work. First, we should compare the tuned secure learned Bloom filter against an untuned secure learned Bloom filter, not just a secure Bloom filter. Second, no efficient implementation of the secure learned Bloom filter exists, so its real-world relevance remains unknown. Third, our tuning algorithm should adapt to different constraint configurations: for example, minimizing expected false positive rate subject to both a memory budget and a maximum false positive rate.

3.3 Reflection

This project strengthened my ability to enter a new subfield independently. I attended several research talks on machine learning security and privacy at Brown University. Although I had no cryptography background, I learned to identify areas where my existing expertise could contribute and where I would be out of my depth. I evaluated several potential projects, recognized that most required background I lacked, and narrowed my focus to tuning secure learned Bloom filters. My experience in machine learning, approximate membership filters and algorithm design made this topic approachable.

This project also developed my appreciation for upfront scoping. Working under a two-week timeline, I did not define objectives or evaluation criteria before starting. As a result, I missed that my tuning algorithm did not address the most important constraint configuration, and that my baseline provided only partial evidence for my claims. Both oversights are now documented as future work, but a clear plan would have helped me catch these missteps sooner.

Finally, this project enhanced my ability formalize and solve optimization problems. I identified and formalize the relevant constraints and objectives, then decomposed the problem into two subproblems. I solved one with calculus and the other with dynamic programming. On its own, each step was straightforward, but identifying the correct steps to take was a challenge.

4 Entropy-Aware Hashing

This project was completed for CSC 519: Computer Networks.

4.1 Background

Hashing is ubiquitous in computing. For example, routing table lookups, intrusion detection systems, load distribution, software defined networking, network address translation and application-layer caches rely on fast hashing [11]. Because hashing is already highly optimized for per-byte throughput, performance gains increasingly come from specialization, where hash functions are tailored

to specific workloads.

Recent works pursue this by exploiting the entropy structure of input data. Entropy learned hashing [8] and `Sede` [9] automatically identify and hash only the high-entropy bytes of keys, reducing unnecessary computation. This approach is theoretically grounded: Mitzenmacher and Vadhan [13] showed that standard hash functions operate by extracting entropy from inputs, so hashing low-entropy bytes wastes work without improving hash quality.

However, both methods specialize incompletely. They treat all keys as equally likely and optimize purely for collision count, ignoring non-uniform key access frequencies and the probe behavior of the target data structure. This potentially misses performance gains for workloads with skewed key distributions and data structure dependent query costs. Therefore, we ask:

1. How can hash functions be automatically specialized to the input distribution, target data structure and underlying hardware architecture architecture simultaneously?

4.2 Summary

4.2.1 Entropy-Aware Hashing

We propose entropy-aware hashing, a framework that automatically specializes a hash function for a given dataset, data structure, and architecture. The framework has two configurable components: the hash function form which determines how data can be read and mixed, and the cost model, which defines what it means for a hash function to be good for a particular workload.

To select which bytes of a key to hash, we extend entropy learned hashing's byte level selection to also support word and double-word granularity. Coarser granularity may include some lower-entropy bytes, but enables more efficient instruction selection, including SIMD and AES-NI instructions that operate on wider data. We also expose alignment as a configurable option, since some architecture penalize unaligned memory accesses. The mixing function is drawn from: MurmurHash3, Thomas Wang's hash, Stafford's hash, XXHash, GXHash, and a single AES round.

We evaluate four cost models. The unweighted collision cost (used by prior work) counts expected collisions uniformly across all keys. The weighted collision cost accounts for non-uniform access frequencies: collisions between frequently accessed keys are penalized more heavily. The maximum weighted probe cost and minimum weighted probe cost measure the expected probe count under the worst- and best-case chain orderings respectively, modeling the cost of collision resolution in separate chaining hash tables.

To specialize, we iterate over hash function forms and greedily select data chunks to hash under the chosen cost model, then benchmark all candidates and select the fastest. We instantiate this framework by specializing for `std::unordered_set` (a separate chaining hash table) on a URL key dataset on the `x64` architecture.

The fastest configuration uses unaligned double-word selection, unweighted collision cost, and a single AES round as the mixing function. This reduces hash latency from 12.1ns (`std::hash`) to 4.5ns, and `std::unordered_set` query latency from 32.9ns to 14.7ns, roughly a 27*times* end-to-end speedup, with no impact on build time. Notably, specialization to key frequency distribution and data structure probe behavior did not provide a speedup: architecture-level specialization via unaligned double-word reads and AES-NI instructions did.

4.2.2 Future Work

We see several promising directions for future work. The most fundamental is theoretical: the hardness of optimal byte subset selection for entropy maximization is uncharacterized, and the greedy heuristic lacks formal approximation guarantees. Establishing these would clarify whether better optimization algorithms are worth pursing.

On the systems side, the framework currently assumes fixed-length keys and a static distribution. Extending to variable length keys is nontrivial. One promising formulation treats alignment of high-entropy regions across keys as a multiple sequence alignment problem. Adapting to shifting distributions would require online entropy estimation to track high-entropy byte positions as the workload evolves.

Finally, the cost model is currently hand-designed, and requires benchmarking. A learned cost model, trained from measurements across datasets, data structures, and architectures, could automatically capture interactions that are difficult to model analytically, enabling better specialization.

4.3 Reflection

This project developed two distinct technical competencies. The first is low-level performance optimization. To identify efficient mixing primitives, I read Intel's `x64` instruction and optimization reference manuals. This led me to the AES-NI extension which includes a single cycle AES round instruction. Learning to reason directly from architecture documentation was a meaningful shift in my optimization approach.

The second is the theory of approximation algorithms. I attempted to prove that finding the maximum-entropy byte subset is **NP-Hard** and bound its best approximation factor via an approximation-preserving reduction. I also tried to show that the greedy algorithm achieves an optimal approximation ratio. I

did not succeed at either. Regardless, the experience was valuable: I worked through standard reduction techniques carefully enough to understand where I failed, and learned about approximation while trying.

References

[1] Ghada Almashaqbeh, Allison Bishop, and Hayder Tirmazi. "Adversary Resilient Learned Bloom Filters". In: *Advances in Cryptology (ASIACRYPT)*. 2026. DOI: 10.1007/978-981-95-5096-8_6.

[2] Burton H. Bloom. "Space/Time Trade-Offs in Hash Coding with Allowable Errors". In: *Commun. ACM* (1970). URL: https://doi.org/10.1145/362686.362692.

[3] Larry Carter et al. "Exact and Approximate Membership Testers". In: *Symposium on Theory of Computing (STOC)*. 1978. URL: https://doi.org/10.1145/800133.804332.

[4] Martin Dietzfelbinger and Stefan Walzer. "Dense Peelable Random Uniform Hypergraphs". In: *European Symposium on Algorithms (ESA)*. 2019. URL: https://doi.org/10.4230/LIPIcs.ESA.2019.38.

[5] Siying Dong et al. "RocksDB: Evolution of Development Priorities in a Key-value Store Serving Large-scale Applications". In: *ACM Transactions Storage* (2021). URL: https://doi.org/10.1145/3483840.

[6] Thomas Mueller Graf and Daniel Lemire. "Binary Fuse Filters: Fast and Smaller Than Xor Filters". In: *Journal of Experimental Algorithmics (JEA)* (2022). URL: http://dx.doi.org/10.1145/3510449.

[7] Thomas Mueller Graf and Daniel Lemire. "Xor Filters: Faster and Smaller Than Bloom and Cuckoo Filters". In: *Journal of Experimental Algorithmics (JEA)* (2020). URL: http://dx.doi.org/10.1145/3376122.

[8] Brian Hentschel, Utku Sirin, and Stratos Idreos. "Entropy-Learned Hashing: Constant Time Hashing with Controllable Uniformity". In: *International Conference on Management of Data (SIGMOD)*. 2022. URL: https://doi.org/10.1145/3514221.3517894.

[9] Renato B. Hoffmann et al. "Automatic Synthesis of Specialized Hash Functions". In: *International Symposium on Code Generation and Optimization (CGO)*. 2025. URL: https://doi.org/10.1145/3696443.3708940.

[10] Tim Kraska et al. "The Case for Learned Index Structures". In: *International Conference on Management of Data (SIGMOD)*. 2018. URL: https://doi.org/10.1145/3183713.3196909.

[11] James F. Kurose and Keith W. Ross. *Computer Networking: A Top-Down Approach*. 9th. Pearson, 2025. ISBN: 9780135429334.

[12] Bruce M. Maggs and Ramesh K. Sitaraman. "Algorithmic Nuggets in Content Delivery". In: *SIGCOMM Comput. Commun. Rev.* (2015). URL: https://doi.org/10.1145/2805789.2805800.

[13] Michael Mitzenmacher and Salil Vadhan. "Why simple hash functions work: exploiting the entropy in a data stream". In: *Symposium on Discrete Algorithms (SODA)*. 2008.

[14] Stefan Walzer. "Peeling Close to the Orientability Threshold: Spatial Coupling in Hashing-Based Data Structures". In: *Symposium on Discrete Algorithms (SODA)*. 2021. URL: https://doi.org/10.1145/3711822.

A Non-Uniform Binary Fuse Filters

A.1 Non-Uniform Binary Fuse Filter Alternative Views

$$A[1] \oplus A[3] \oplus A[5] \oplus A[7] = 100$$

$$A[3] \oplus A[5] \oplus A[8] \oplus A[0] = 011$$

$$A[8] \oplus A[9] \oplus A[13] \oplus A[14] = 110$$

$$A[10] \oplus A[13] \oplus A[15] \oplus A[0] = 111$$

Figure 9: *System of linear equations with variables in $\mathbb{F}_2^3$ induced by Figure 5. Each $A[i]$ is the i-th entry in the backing array A (zero-indexed). The keys are x, y, w and z with fingerprints 100, 011, 110 and 111, respectively. Array entries and fingerprints are marked with corresponding key color(s). The phantom entry is colored gray. There is one equation for each key.*

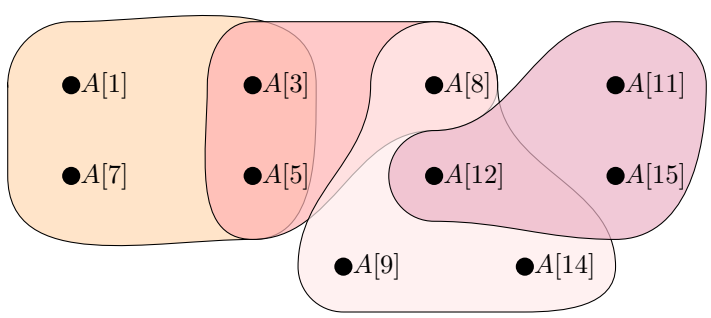

Figure 10: *Hypergraph model of Figure 9. Each $A[i]$ is the i-th entry in the backing array A (zero-indexed). Hyperedges correspond to keys x, y, w and z, colored accordingly. Vertices of degree zero and phantom entry $A[0]$ not shown.*

A.2 Non-Uniform Binary Fuse Filter Query Algorithms

Algorithm 2 Conditional branch algorithm.

```
function Contains(k)
    f ← Fingerprint(k)
    f′ ← A[Hash1(k)] ⊕ A[Hash2(k)] ⊕ A[Hash3(k)]
    if IsFourEdge(k) then
        f′ ← f′ ⊕ A[Hash4(k)]
    end if
    return f == f′
end function
```

Algorithm 3 Branchless algorithm.

```
function Contains(k)
    f ← Fingerprint(k)
    f′ ← A[Hash1(k)] ⊕ A[Hash2(k)] ⊕ A[Hash3(k)]
    f′ ← f′ ⊕ (-IsFourEdge(k) & A[Hash4(k)])
    return f == f′
end function
```

Algorithm 4 Phantom entry algorithm.

```
function Contains(k)
    f ← Fingerprint(k)
    f′ ← A[Hash1(k) + 1] ⊕ A[Hash2(k) + 1] ⊕ A[Hash3(k) + 1]
    f′ ← f′ ⊕ A[-IsFourEdge(k) & (Hash4(k) + 1)]
    return f == f′
end function
```